Occurence

Updated: 2018-09-06

An event anchored at a specified moment in time composed of two actions: one when time goes forward, and another when time goes backward. The latter is most often used to revert the former.

Properties

```
float time { get; }
```

The time in seconds on the parent timeline at which the occurrence will happen.

```
bool repeatable { get; }
```

Indicates whether this occurrence can happen more than once; that is, whether it will stay on the timeline once it has been rewound.

Methods

```
abstract void Forward()
```

The action that is executed when time goes forward.

```
abstract void Backward()
```

The action that is executed when time goes backward.

Triggers

The following methods are all called from a `Timeline` instance. They are placed in this section for the sake of organization only.

```
Occurrence Schedule(float time, bool repeatable, Occurrence occurrence)
Occurrence Schedule(float time, bool repeatable, ForwardFunc<T> forward, BackwardFunc<T> b
```

```
Occurrence Schedule(float time, bool repeatable, ForwardAction forward, BackwardAction bac
Occurrence Schedule(float time, ForwardAction forward)
```

Schedules an occurrence at a specified absolute time in seconds on the timeline.

```
Occurrence Do(bool repeatable, Occurrence occurrence)
Occurrence Do(bool repeatable, ForwardFunc<T> forward, BackwardFunc<T> backward)
Occurrence Do(bool repeatable, ForwardAction forward, BackwardAction backward)
```

Executes an occurrence now and places it on the schedule for rewinding.

```
Occurrence Plan(float delay, bool repeatable, Occurrence occurrence)
Occurrence Plan(float delay, bool repeatable, ForwardFunc<T> forward, BackwardFunc<T> back
Occurrence Plan(float delay, bool repeatable, ForwardAction forward, BackwardAction backwa
Occurrence Plan(float delay, ForwardAction forward)
```

Plans an occurrence to be executed in the specified delay in seconds.

```
Occurrence Memory(float delay, bool repeatable, Occurrence occurrence)
Occurrence Memory(float delay, bool repeatable, ForwardFunc<T> forward, BackwardFunc<T> ba
Occurrence Memory(float delay, bool repeatable, ForwardAction forward, BackwardAction back
Occurrence Memory(float delay, ForwardAction forward)
```

Creates a "memory" of an occurrence at a specified "past-delay" in seconds. This means that the occurrence will only be executed if time is rewound, and that it will be executed backward first.

If `repeatable` is set to `false`, the occurrence will be cancelled after it has been rewound. This is useful for code that occurs as a result of object interaction and would therefore reoccur by itself after a rewind.

All the trigger methods return the created occurrence as a result, in case you need to cancel or reschedule it via the methods below.

```
void Cancel(Occurrence occurrence)
```

Removes the specified occurrence from the timeline.

```
bool TryCancel(Occurrence occurrence)
```

Removes the specified occurrence from the timeline and returns `true` if it is found. Otherwise, returns `false`.

```
void Reschedule(Occurrence occurrence, float time)
```

Changes the absolute time in seconds of the specified occurrence on the timeline.

```
void Postpone(Occurrence occurrence, float delay)
```

Moves the specified occurrence forward on the timeline by the specified delay in seconds.

```
void Prepone(Occurrence occurrence, float delay)
```

Moves the specified occurrence backward on the timeline by the specified delay in seconds.

Examples

Changing the color of the GameObject to blue 5 seconds after Space is pressed. This delay will take into consideration pauses, fast-forwards and slow-downs. However, this occurrence is not (yet) rewindable: the object will not go back to its original color on rewind.

```
using UnityEngine;
using Chronos;

class MyBehaviour : MonoBehaviour
{
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            GetComponent<Timeline>().Plan(5, ChangeColor);

            // Notice the absence of () after the method name.
            // This is because we are refering to the method itself,
            // not calling it right now.
        }
    }

    void ChangeColor()
    {
        GetComponent<Renderer>.material.color = Color.blue;
    }
}
```

Same example as before, but this time, we pass a color parameter to our method:

```
using UnityEngine;
using Chronos;

class MyBehaviour : MonoBehaviour
{
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
```

```
            GetComponent<Timeline>().Plan(5, delegate { ChangeColor(Color.red); });

            // Here, we create a delegate (an inline method) to
            // be called in 5 seconds. In turn, our delegate calls
            // the ChangeColor with the color red as a parameter.
        }
    }

    void ChangeColor(Color newColor)
    {
        GetComponent<Renderer>.material.color = newColor;
    }
}
```

If you are unfamiliar with delegates, it is recommended that you take a look at the MSDN tutorial on delegates . Note, however, that you won't have to create your own delegate types to use Chronos — you will only need a basic understanding of what they are and how they can be created.

Now, let's make our color change a rewindable occurrence. When time is rewound up to the moment the GameObject became blue, it should automatically revert back to its original color.

```
using UnityEngine;
using Chronos;

class MyBehaviour : MonoBehaviour
{
    void Start()
    {
        GetComponent<Timeline>().Plan
        (
            5, // In 5 seconds...

            true, // ... create a repeatable event...

            delegate // ... that sets the color to blue and saves the previous one...
            {
                Renderer renderer = GetComponent<Renderer>();
                Color previousColor = renderer.material.color;
                renderer.material.color = Color.blue;
                return previousColor; // This will be passed as the first parameter below
            },

            delegate (Color previousColor) // ... and reverts it on rewind.
            {
                Renderer renderer = GetComponent<Renderer>();
                renderer.material.color = previousColor;
            }
        );

        // You can pass any object, be it a simple integer or
        // a complex class, from the first delegate
        // to the second. This allows to indicate a 'state'
        // to which you can easily revert.
```

```
    }
}
```

Try experimenting with the `repeatable` parameter from that example to get an understanding of what it does. For example, try setting it to `false`, then rewinding time after the object turned blue until it goes back to its original color, then letting time go forward normally. You'll realize it doesn't turn blue again! That's because the occurrence was removed from the timeline after rewinding.

That previous example works, but it's a bit tedious to set up. Imagine that we often wanted to have rewindable color-change occurrences like this one — it would be quite annoying to type that code every time! Fortunately, we don't have to.

In the following example, we'll create our own `Occurrence` class and transform our previously lengthy code into a reusable one-liner.

```
using UnityEngine;
using Chronos;

// Inherit Occurrence and implement Forward() and Backward()
public class ChangeColorOccurrence : Occurrence
{
    Material material;
    Color newColor;
    Color previousColor;

    public ChangeColorOccurrence(Material material, Color newColor)
    {
        this.material = material;
        this.newColor = newColor;
    }

    public override void Forward()
    {
        previousColor = material.color;
        material.color = newColor;
    }

    public override void Backward()
    {
        material.color = previousColor;
    }

    // You can create occurrences of any complexity!
    // This simple one is only for the sake of a short example.
}

class MyBehaviour : MonoBehaviour
{
    void Start()
    {
```

```
        Timeline time = GetComponent<Timeline>();
        Material material = GetComponent<Renderer>().material;

        // Change the color to blue in 5 seconds
        time.Plan(5, true, new ChangeColorOccurrence(material, Color.blue));

        // Change the color to red in 10 seconds
        time.Plan(10, true, new ChangeColorOccurrence(material, Color.red));
    }
}
```

Finally, if you don't need to pass data between the forward and backward actions, you can simply return nothing in the first delegate and have no parameter in the second:

```
using UnityEngine;
using Chronos;

class MyBehaviour : MonoBehaviour
{
    void Start()
    {
        GetComponent<Timeline>().Plan
        (
            5, // In 5 seconds...

            true, // ... create a repeatable event...

            delegate
            {
                Debug.Log("Going forward!");
            },

            delegate
            {
                Debug.Log("Going backward!");
            }
        );
    }
}
```

You now have the tools to make any kind of custom code work with Chronos; whether time flows normally, slower, faster or even backwards!

When making rewindable occurrences, it is crucial to remember that if your code occurs from the interaction between game objects (e.g. collisions), it should — in almost all cases — **not** be set to `repeatable`.

For example, say you change the color of two objects to red when they collide (in `OnCollisionEnter`). If that occurrence is repeatable and you rewind, then let time run normally, not only will the objects change to red from the existing occurrence, but they'll collide *again*, creating a new occurrence. This can quickly lead to unexpected results, so be careful!

Previous

TimelineChild

A component that transfers the effects of any ancestor `Timeline` in the hierarchy to the current GameObject.

Next

Recorder

An abstract base component that saves snapshots of any kind at regular intervals to enable rewinding.

Was this article helpful?

Be the first to vote!

Yes, helpful | No, not for me